

YT-DLP Container Project



Run `yt-dlp` inside a container with optional VPN support. This project provides a complete Docker/Podman-based solution for downloading videos from YouTube and other supported sites, featuring:

- **Dual Runtime Support:** Works with both Podman (preferred) and Docker
- **VPN Integration:** Route downloads through OpenVPN for privacy
- **Landing Page:** Seamless cookie authentication flow for YouTube
- **Web Interface:** MeTube provides a clean, modern web UI
- **Angular Dashboard:** Modern standalone dashboard on port 9090
- **Cookie Authentication:** Built-in support for YouTube browser cookies
- **CLI Access:** Direct `yt-dlp` command-line access
- **Batch Processing:** Download from URL lists and channel subscriptions
- **Multi-Service:** Designed to work alongside JDownloader

Table of Contents

- [Quick Start](#)
- [Installation](#)
- [Configuration](#)
- [Usage](#)
- [Container Runtime](#)
- [VPN Setup](#)
- [Advanced Usage](#)
- [Testing](#)
- [Troubleshooting](#)
- [Contributing](#)
- [Documentation](#)
- [License](#)

Quick Start

```
# 1. Clone the repository (with submodules: constitution/,
    Challenges/)
git clone --recursive https://github.com/milos85vasic/YT-DLP.git
cd YT-DLP
# Already cloned without --recursive? Run: git submodule update --
    init --recursive
# (./init also self-heals this automatically.)

# 2. Copy and edit configuration
cp .env.example .env
```

```
# Edit .env with your settings

# 3. Initialize and start
./init
./start

# 4. Access the web interface
# Open http://localhost:9090 for the Angular Dashboard
# Or http://localhost:8086 for the Landing Page
```

Installation

Prerequisites

Required: One of the following container runtimes: - Podman (recommended, rootless by default) - Docker

Optional: - podman-compose or docker-compose for compose operations - OpenVPN configuration file (for VPN support)

Setup Steps

1. Clone the repository (with submodules):

```
git clone --recursive https://github.com/milos85vasic/YT-DLP.git
cd YT-DLP
```

This repo uses git submodules (constitution/ — the Helix Universal Constitution this project inherits — and Challenges/). The --recursive flag populates them on clone. If you already cloned without it, run `git submodule update --init --recursive`. `./init` also self-heals this.

2. Create configuration:

```
cp .env.example .env
```

3. Edit .env with your settings:

```
# Required settings
USE_VPN=false # Set to true to enable VPN
DOWNLOAD_DIR=/path/to/downloads # Where videos will be saved

# VPN settings (if USE_VPN=true)
VPN_USERNAME=your_username
VPN_PASSWORD=your_password
VPN_OVPN_PATH=/path/to/config.ovpn
```

4. Initialize the environment:

```
./init
```

This creates necessary directories and validates your configuration.

5. Start the services:

```
./start
```

Configuration

Environment Variables

Variable	Required	Default	Description
USE_VPN	Yes	false	Enable VPN routing (true/false)
DOWNLOAD_DIR	Yes	-	Absolute path to download directory
VPN_USERNAME	If VPN	-	VPN account username
VPN_PASSWORD	If VPN	-	VPN account password
VPN_OVPN_PATH	If VPN	-	Path to OpenVPN config file
CONTAINER_RUNTIME	No	auto	Force runtime: podman, docker, or auto
METUBE_PORT	No	8086	Web interface port
YTDLP_VPN_PORT	No	3130	VPN proxy port
TZ	No	Europe/Moscow	Timezone for containers
SERVICE_MODE	No	false	Enable automatic processing

Example .env (No VPN)

```
USE_VPN=false
DOWNLOAD_DIR=/home/user/Downloads/Videos
METUBE_PORT=8086
TZ=America/New_York
```

Example .env (With VPN)

```
USE_VPN=true
DOWNLOAD_DIR=/home/user/Downloads/Videos
VPN_USERNAME=myvpnuser
VPN_PASSWORD=myvpnpass
VPN_OVPN_PATH=/home/user/vpn/config.ovpn
METUBE_PORT=8086
TZ=America/New_York
```

Usage

Management Scripts

Script	Description
<code>./init</code>	Initialize environment, validate config
<code>./start</code>	Start services (uses VPN setting from <code>.env</code>)
<code>./start_no_vpn</code>	Start services without VPN
<code>./stop</code>	Stop all services
<code>./restart</code>	Stop and restart services
<code>./update-images</code>	Pull latest container images
<code>./setup-auto-update</code>	Setup automatic updates (Podman cron)
<code>./status</code>	Show service status and container info
<code>./check-vpn</code>	Verify VPN connection status
<code>./cleanup [all\ ytdlp\ jdownloader]</code>	Remove containers
<code>./download <URL></code>	Download a video
<code>./download --batch</code>	Download from <code>urls.txt</code>
<code>./download --channels</code>	Download from <code>channels.txt</code>

Web Interface (Metube)

Once running, access the services at:

- **YT-DLP Dashboard (new):** <http://localhost:9090> — Modern Angular UI with queue, history, and download management
- **Landing Page:** <http://localhost:8086> — Cookie authentication flow
- **MeTube UI:** <http://localhost:8088> — Original MeTube web interface

The Landing Page handles cookie authentication and redirects to MeTube when ready.

Landing Page Features: - Automatic YouTube cookie status check - Step-by-step guide for cookie export - Drag-and-drop cookie upload - Auto-redirect to MeTube when authenticated

YT-DLP Dashboard (at <http://localhost:9090>): - Modern Angular standalone application - Add downloads with quality/format selection — **form re-enables immediately after `/api/add` returns ok**, so you can submit the next URL while the previous is still tracking - Real-time queue with per-state visuals: pending / preparing / downloading / postprocessing / finished / error, each with a distinct icon, badge colour, and live progress bar (with shimmer when active and indeterminate animation when no percent is yet reported) - **Cancel-from-Queue confirmation dialog** — destructive cancel always prompts before firing - **Aborted-history visibility** — cancelled queue items show up on the History page with status aborted (vendor MeTube would have silently dropped them; the dashboard records

them via the new `/api/aborted-history` landing endpoint) - 16 supported platforms with per-platform status badges - VPN-state pill in the navbar - Cookie management page with per-platform freshness breakdown - **Bulk operations on Queue and History**: per-row checkboxes, Select-all in the toolbar, and dedicated **Clear All / Delete All** buttons (both with confirmation; *Delete* requires ticking an acknowledgement checkbox before it can fire) - Dark theme UI

See [USER GUIDE → Managing Queue and History](#) for the full bulk-action workflow including how the modal acknowledgement gating works.

MeTube UI (at <http://localhost:8088>): - Paste URLs to download - View download queue and progress - Select format/quality - Playlist and channel support - Download history

Command Line Usage

Download a single video:

```
./download 'https://www.youtube.com/watch?v=VIDEO_ID'
```

Download with options:

```
# With Podman (auto-detected)
```

```
./download 'URL' -f 'bestvideo[height<=720]+bestaudio'
```

```
# Or directly with container runtime
```

```
podman exec yt-dlp-cli yt-dlp 'URL'
```

```
docker exec yt-dlp-cli yt-dlp 'URL'
```

Batch downloads:

```
# Add URLs to ./yt-dlp/config/urls.txt
```

```
echo 'https://youtube.com/watch?v=VIDEO1' >> ./yt-dlp/config/urls.txt
```

```
echo 'https://youtube.com/watch?v=VIDEO2' >> ./yt-dlp/config/urls.txt
```

```
# Download all
```

```
./download --batch
```

Channel subscriptions:

```
# Add channel URLs to ./yt-dlp/config/channels.txt
```

```
echo 'https://youtube.com/c/ChannelName' >> ./yt-dlp/config/channels.txt
```

```
# Download recent videos (last 7 days)
```

```
./download --channels
```

Viewing Logs

```
# All services
```

```
podman-compose logs -f
```

```
# or
```

```
docker-compose logs -f
```

```
# Specific service
```

```
podman-compose logs -f yt-dlp-cli
```

Container Runtime

This project automatically detects and uses the available container runtime:

Podman (Recommended)

```
# Check installation
```

```
podman --version
```

```
# The scripts will automatically use Podman if available
```

```
./start # Uses Podman by default
```

Benefits of Podman: - **Rootless by default**: Runs without root privileges - **Daemonless**: No background service required - **Docker-compatible**: Drop-in replacement - **Systemd integration**: Native systemd support

Docker

```
# Check installation
```

```
docker --version
```

```
# If Podman is not installed, scripts will use Docker
```

```
./start # Uses Docker if Podman unavailable
```

Forcing a Runtime

To force a specific runtime, edit `.env`:

```
CONTAINER_RUNTIME=podman # Force Podman
```

```
# or
```

```
CONTAINER_RUNTIME=docker # Force Docker
```

Compose Commands

With Podman:

```
# Using podman-compose (standalone)
```

```
podman-compose --profile vpn up -d
```

```
podman-compose --profile no-vpn up -d
```

```
# Using podman compose (built-in)
```

```
podman compose --profile vpn up -d
```

With Docker:

```
# Using docker-compose (standalone)
docker-compose --profile vpn up -d

# Using docker compose (plugin)
docker compose --profile vpn up -d
```

VPN Setup

Supported VPN Providers

Any OpenVPN-compatible provider should work. Tested with: - Private Internet Access (PIA) - NordVPN - Mullvad - ProtonVPN

Configuration Steps

1. **Download OpenVPN config** from your VPN provider
2. **Place config file** in a secure location (e.g., ~/vpn/config.ovpn)
3. **Update .env:**

```
USE_VPN=true
VPN_USERNAME=your_username
VPN_PASSWORD=your_password
VPN_OVPN_PATH=/home/user/vpn/config.ovpn
```

4. **Initialize:**

```
./init
# This creates vpn-auth.txt from your credentials
```

5. **Start with VPN:**

```
./start
```

6. **Verify connection:**

```
./check-vpn
```

VPN Authentication

The init script will: - Create vpn-auth.txt with your credentials - Set permissions to 600 (readable only by owner) - Add auth-user-pass directive to OpenVPN config if missing

Important: Never commit vpn-auth.txt to git. It's already in .gitignore.

Automatic Updates

All container images are automatically kept up-to-date through two mechanisms:

1. On Every Start

Whenever you start the services, the latest images are pulled automatically:

```
./start          # Pulls latest images before starting
./start_no_vpn   # Also pulls latest images
```

2. Periodic Background Updates (Every 3-4 Hours)

Docker Users: Watchtower is included and configured to check for updates every 3 hours. It will automatically pull new images and restart containers when updates are available.

Podman Users: Since Watchtower requires Docker socket access, Podman users should set up a cron job:

```
# Setup automatic updates
./setup-auto-update

# This creates a cron job that runs every 3 hours
# Logs are saved to ./logs/update.log
```

Manual Updates

To manually check for and pull updates:

```
./update-images
```

Or specify the runtime:

```
CONTAINER_RUNTIME=docker ./update-images
```

Updated Images

The following images are automatically updated: - **Metube:** ghcr.io/alexta69/metube:latest - **yt-dlp CLI:** th3a/yt-dlp:latest - **OpenVPN:** dperson/openvpn-client:latest - **Watchtower:** containrrr/watchtower:latest

Advanced Usage

Custom yt-dlp Options

Edit ./yt-dlp/config/yt-dlp.conf:

```
# Video quality
-f "bestvideo[height<=720]+bestaudio/best"

# Output template
-o "/downloads/(uploader)s/(title)s.%(ext)s"
```



```
# Subtitles
--write-sub
--sub-langs en,es
--embed-sub

# Metadata
--add-metadata
--embed-thumbnail
```

Using Cookies

YouTube requires browser cookies to download videos. The easiest method is using the Landing Page:

1. **Open <http://localhost:8086>** - The Landing Page will guide you
2. **Export cookies** from your browser using “Get cookies.txt LOCALLY” extension
3. **Upload cookies** via drag-and-drop on the Landing Page
4. **Auto-redirect** to MeTube when authenticated

Alternatively, to access age-restricted or subscriber-only content:

1. **Export cookies** from your browser using an extension
2. **Place cookies file** at `./yt-dlp/cookies/youtube_cookies.txt`
3. Restart: `./stop && ./start`

Cookie Helper Scripts:

```
# Interactive cookie setup
./yt-dlp/cookies/setup-cookies.sh
```

Service Mode

Enable automatic processing of URLs and channels:

```
# In .env
SERVICE_MODE=true
```

When enabled, containers will automatically process: - `./yt-dlp/config/urls.txt` (batch downloads) - `./yt-dlp/config/channels.txt` (channel subscriptions)

Integration with JDownloader

This project is designed to work alongside [JDownloader](#):

- Separate VPN containers for each service
- Different external IP addresses (if VPN enabled)
- Non-conflicting ports

To use together:

```
# Start both services
./start # Starts YT-DLP
# (Start JDownloader separately in its directory)
```

Testing & Verification

This project uses a **4-Gate verification system** to prevent the universal “green tests, broken product” problem.

Quick Validation

```
# One command – run everything before pushing
make dev-check

# Or individual gates:
make smoke # E2E smoke tests against real services
make audit # Test suite quality audit (score: 70/70)
make test # Full test suite (121 tests)
```

The 4 Gates

No task is complete until all gates pass:

Gate	What	Command
1. Contract	API matches OpenAPI spec	contracts/metube-api.openapi.yaml
2. Integration	Real HTTP calls, no mocks	./tests/test-integration-realhttp.sh
3. Smoke	E2E against running containers	./scripts/smoke-test.sh
4. Manual	Human verifies in dashboard at :9090	—

Test Suites

```
# Run all tests
./tests/run-comprehensive-tests.sh

# Run specific categories
./tests/run-tests.sh -p unit # Unit tests (pure logic only)
./tests/run-tests.sh -p integration # Integration tests
./tests/run-tests.sh -p scenario # Scenario tests
./tests/run-tests.sh -p error # Error tests

# Real HTTP integration tests (no mocks – hits actual services)
./tests/test-integration-realhttp.sh
```

```
# E2E smoke tests (requires running containers)
```

```
./scripts/smoke-test.sh
```

```
# Test quality audit (0-70 score)
```

```
./scripts/test-audit.sh
```

Current Results: - 121 tests passing (0 failures) - 22/22 smoke tests passing - 22/22 real HTTP integration tests passing - Test audit score: 70/70 - 11 tests skipped (known platform restrictions)

See [TEST_RESULTS.md](#) for detailed results. See [docs/VERIFICATION.md](#) for the full verification philosophy.

Troubleshooting

Container Runtime Not Found

```
# Check if Podman is installed
```

```
which podman
```

```
podman --version
```

```
# Or Docker
```

```
which docker
```

```
docker --version
```

```
# Install Podman (recommended)
```

```
# Fedora/RHEL: sudo dnf install podman podman-compose
```

```
# Ubuntu/Debian: sudo apt-get install podman podman-compose
```

```
# Arch: sudo pacman -S podman podman-compose
```

VPN Connection Issues

```
# Check VPN status
```

```
./check-vpn
```

```
# View VPN logs
```

```
podman-compose logs -f openvpn-yt-dlp
```

```
# Common fixes:
```

```
# 1. Verify VPN credentials in .env
```

```
# 2. Check OpenVPN config path
```

```
# 3. Ensure auth-user-pass directive in .ovpn file
```

Permission Denied Errors

```
# Fix directory permissions
```

```
chmod -R 755 ./yt-dlp ./metube
```

```
# For rootless Podman, ensure your user owns the download
directory
sudo chown -R $USER:$USER /path/to/downloads
```

Download Directory Not Found

The init script will prompt to create the directory. To create manually:

```
mkdir -p /path/to/downloads
```

Port Already in Use

Change the port in `.env`:

```
METUBE_PORT=8087 # Use different port
```

YouTube Downloads Failing - “No video formats found” or Bot Detection

This is caused by YouTube’s bot detection. YouTube requires cookies from a browser session to download videos.

Quick Fix:

```
# Run the cookie setup helper
./yt-dlp/cookies/setup-cookies.sh

# Or manually:
# 1. Install "Get cookies.txt LOCALLY" extension for Firefox/
    Chrome
# 2. Go to youtube.com (logged in)
# 3. Export cookies as youtube_cookies.txt in ./yt-dlp/cookies/
# 4. Restart: ./stop && ./start_no_vpn
```

Detailed instructions: See [docs/YOUTUBE_DOWNLOAD_FIX.md](#)

Troubleshooting: - Ensure cookies file exists: `cat ./yt-dlp/cookies/youtube_cookies.txt` - Check container logs: `podman logs metube-direct | grep -i error` - Cookies may expire - re-export periodically

Port Reference

Port	Service	Description
8086	Landing Page	Cookie authentication & redirect (No VPN)
8087	Landing Page	Cookie authentication & redirect (VPN)
8088	Metube	Web interface for yt-dlp (No VPN)
9090	Dashboard	Angular dashboard for yt-dlp (No VPN)
8081	Metube API	Internal API (container)

Port	Service	Description
3130	yt-dlp VPN	VPN proxy port
3129	JDownloader VPN	VPN proxy (if using JDownloader)
5800	JDownloader	Web UI (if using JDownloader)
5900	JDownloader	VNC (if using JDownloader)

Dual-version media pipeline (webready video + mp3)

A dedicated `media_postprocessor` sidecar (Python, no-vpn profile) produces a **guaranteed-playable derivative alongside every download**, while keeping the **original file untouched as a zero-loss master**:

- Every downloaded **video** → `webready-<base>.mp4`, transcoded to **H.264 + AAC with faststart** (the `moov` atom is muxed before `mdat` so playback can start before the whole file downloads) — built for Android-TV playback. Audio that is already AAC ≤2ch is copied losslessly; otherwise it is re-encoded to AAC. (`media_postprocessor/transcoder.py::transcode_video`)
- Every downloaded **audio** file → `<base>.mp3` at **320 kbps CBR**. (`media_postprocessor/transcoder.py::derive_mp3`)
- The original download is never modified — it is the zero-loss master. The `webready .mp4` is a re-encode and is **not** bit-exact (see the design spec's honesty clause); the master is the lossless artifact.

Each derivative is written atomically (`ffmpeg` writes a `<final>.partial` temp file and only `os.replace()`s it onto the final path on `ffmpeg` exit 0) and then **ffprobe-validated** (video: h264 + aac + non-zero duration + faststart; audio: mp3 + non-zero duration) before the job is marked done — a failed or interrupted run never leaves a half-written or invalid output. (`media_postprocessor/transcoder.py`)

How it runs

The sidecar's `main()` orchestration (`media_postprocessor/service.py::main`) starts in this order:

1. Init the SQLite (WAL) jobs DB and **re-queue any job a crashed worker left running** so interrupted work resumes to a valid, complete output.
2. **Backfill** — a one-time startup full-scan of the whole library (`watcher.reconcile_scan`) enqueues every existing video/audio file that has no derivative yet.
3. **Watch** — a watchdog `PollingObserver` (polling is used because `inotify` is unreliable on network mounts) enqueues new downloads as they land.
4. A background **worker thread** drains the queue, deriving each output and marking the job done (with its output path) or failed (with the error).
5. A **periodic reconcile** thread re-scans the library every `MP_RECONCILE_INTERVAL` seconds — the safety net for files the watcher deferred as still-being-written.
6. A small stdlib **HTTP status server** on `MP_PORT`.

First start backfills your existing library. On the very first run the postprocessor transcodes every video/audio file already in `${DOWNLOAD_DIR}` to produce the missing `webready-*.mp4 / *.mp3` derivatives. Expect this initial backfill pass — it can take a while on a large library and runs at `MP_MAX_CONCURRENCY`.

Status & dashboard. The dashboard proxies `/api/postprocess/*` to the sidecar (`dashboard/nginx.conf.template`); `GET /api/postprocess/status` returns per-state job counts (`queued/running/done/failed/canceled`) and `GET /api/postprocess/jobs` lists each job. The dashboard surfaces these states so you can see derivation progress (`media_postprocessor/service.py`).

Mid-write protection: a freshly-written file whose mtime is younger than `MP_MIN_STABLE_AGE` seconds is deferred (it may still be growing as `yt-dlp/ffmpeg` finish merging); the periodic reconcile picks it up once it has settled (`media_postprocessor/watcher.py`). In-progress (`*.part`, `*.ytdl`, `*.partial`) files and the pipeline's own outputs (`webready-` prefix, any `.mp3`) are never re-processed (`media_postprocessor/media_probe.py::classify_target`).

Environment variables

Variable	Default	Purpose
<code>DOWNLOAD_DIR</code>	<code>/downloads</code> (container)	Library directory the postprocessor watches, backfills, and writes derivatives into (<code>config.py</code>)
<code>MP_PORT</code>	<code>8089</code>	Port for the status HTTP server (<code>config.py</code>); the dashboard proxies <code>/api/postprocess/*</code> here
<code>MP_MAX_CONCURRENCY</code>	<code>1</code> (compose sets <code>2</code>)	Worker concurrency cap (<code>config.py</code>)
<code>MP_MIN_STABLE_AGE</code>	<code>10</code> (seconds)	Mid-write guard — a file is enqueued only after its mtime has been stable this long (<code>watcher.py</code>)
<code>MP_RECONCILE_INTERVAL</code>	<code>30</code> (seconds)	How often the periodic reconcile re-scans the library (<code>service.py</code>)

The `docker-compose.yml` `media_postprocessor` service runs under the `no-vpn` profile with `mem_limit: 1g` and `oom_score_adj: 1000` — it is intentionally the **first container sacrificed under memory pressure**, never the download/queue services.

Documentation

- **[USER GUIDE.md](#)** - Complete user manual with examples
- **[TEST RESULTS.md](#)** - Comprehensive test results report

- **AGENTS.md** - Development guide and coding standards
- **CONTRIBUTING.md** - Contribution guidelines
- **docs/YOUTUBE_DOWNLOAD_FIX.md** - YouTube cookie authentication guide
- **tests/README.md** - Testing documentation

Contributing

We welcome contributions! Please see [CONTRIBUTING.md](#) for guidelines.

License

This project is licensed under the GNU General Public License v3.0 - see the [LICENSE](#) file for details.

Acknowledgments

- [yt-dlp](#) - The video downloader
- [Metube](#) - Web UI for yt-dlp
- [Podman](#) - Container engine
- [OpenVPN](#) - VPN solution

Support

- **Issues:** [GitHub Issues](#)
- **Discussions:** [GitHub Discussions](#)

Disclaimer: This tool is for personal use only. Respect copyright laws and terms of service of content providers.